VELA Final-Version Methodology and Development Plan

Prepared from the consolidated design discussion

March 9, 2026

1 Purpose and Scope

VELA (**V**isco**E**lastic **L**inear-response **A**nalyzer) is intended to become an equilibrium-stress based linear viscoelastic inference tool for molecular simulation data. The final target is not merely "stress autocorrelation to modulus" conversion, but a branch-aware, transform-aware, and uncertainty-aware framework that distinguishes what is directly identifiable from the supplied data and what is not.

The central design principle is:

> **VELA should first determine what quantity is identifiable from the input, and only then transform or infer frequency-domain rheological observables.**

This document consolidates the full software and methodology plan, including the current correction phase, the reduced/absolute branch logic, the μ_A pathways, the transform stack, and the long-range roadmap toward the final CAFT-Bounds method.

2 Scientific Basis

2.1 Why the current community practice needs refinement

A common equilibrium formula for the shear relaxation modulus is written schematically as

$$G(t) = \frac{V}{k_B T}\langle \sigma(0)\sigma(t)\rangle, \tag{1}$$

which is frequently used for liquids, melts, and systems that clearly reach terminal flow within the simulated time window. This logic is also the current conceptual basis of the public VELA repository.

However, for solids, glasses, permanently crosslinked networks, and slow systems with nonzero equilibrium shear modulus, the relation between the true relaxation modulus and the fixed-strain stress autocorrelation requires more care. Wittmer and co-workers showed that, for the solid case,

$$G(t) = C_\tau(t) = C_\gamma(t) + G_{\mathrm{eq}}, \tag{2}$$

where $C_\gamma(t)$ is the fixed-strain stress autocorrelation and G_{eq} is the static equilibrium modulus. Therefore, a fixed-strain stress autocorrelation alone does not, in general, determine the absolute plateau modulus. This is the key reason to distinguish *reduced* and *absolute* branches in VELA.[1]

[1] J. P. Wittmer et al., "Shear-stress relaxation and ensemble transformation of shear-stress autocorrelation functions," *Phys. Rev. E* **91**, 062306 (2015), https://arxiv.org/abs/1508.03730.

A complementary practical identity used by Wittmer and Kriuchevskyi is

$$G(t) = \mu_A - h(t), \tag{3}$$

where μ_A is the affine shear elasticity and $h(t)$ is a rescaled mean-square displacement of the instantaneous shear stress. This identity provides the basis for the absolute branch when μ_A is available.[2]

2.2 Reduced vs absolute modulus

VELA will formalize the following distinction:

- **Reduced branch**: the quantity directly identifiable from fixed-strain equilibrium stress fluctuations when no affine modulus information is supplied.
- **Absolute branch**: the fully anchored modulus obtained when an absolute elastic reference is provided, most naturally through μ_A.

In stress-only operation, the safe output is

$$G_{\mathrm{red}}(t) \equiv C_\gamma(t), \tag{4}$$

and in frequency space

$$G'_{\mathrm{red}}(\omega) = G'(\omega) - G_{\mathrm{eq}}, \qquad G''(\omega) = G''(\omega). \tag{5}$$

When μ_A is supplied, VELA can reconstruct the absolute branch. In the current raw-output convention used by the repository, this is implemented through

$$\mu_{F,\mathrm{raw}} = G_{\mathrm{red,raw}}(0), \qquad G_{e,\mathrm{raw}} = \mu_{A,\mathrm{raw}} - \mu_{F,\mathrm{raw}}, \tag{6}$$

and thus

$$G_{\mathrm{abs,raw}}(t) = G_{\mathrm{red,raw}}(t) + G_{e,\mathrm{raw}}. \tag{7}$$

3 Current Repository Status and Immediate Corrections

The public VELA repository currently contains the following top-level implementation pieces:

- `main.cc`
- `correlator.h`
- `correlator_mode0.cc`
- `correlator_mode1.cc`
- `correlator_Gw.cc`
- `Makefile`

The current README describes mode 0 as using `xy,xz,yz` and mode 1 as using `xx,yy,zz,xy,xz,yz`, with mode 1 theoretically defined in terms of normal-stress differences.[3]

The first correction phase must remain a *minimal-change* update on top of the present codebase.

[2] S. Kriuchevskyi et al., "Computing the shear-stress relaxation modulus from the shear-stress mean-square displacement," *Phys. Rev. E* **93**, 012103 (2016), DOI: 10.1103/PhysRevE.93.012103.

[3] VELA public repository, https://github.com/haoyuwu97/VELA.

3.1 Correction 1: mode 1 must use normal-stress differences

The README formula already states that mode 1 is based on

$$N_{xy} = \sigma_{xx} - \sigma_{yy}, \qquad N_{xz} = \sigma_{xx} - \sigma_{zz}, \qquad N_{yz} = \sigma_{yy} - \sigma_{zz}, \tag{8}$$

combined with the three shear stresses. The current public code path in `main.cc` passes raw xx, yy, zz instead of these differences. This must be corrected first.

3.2 Correction 2: centered correlations must become the scientific default

The current repository already exposes a `norm` switch inside `evaluate_mode0()` and `evaluate_mode1()`, but the main program does not enable it. The correction phase sets centered correlations as the default scientific path:

$$\langle \delta\sigma(0)\, \delta\sigma(t) \rangle \tag{9}$$

rather than raw $\langle \sigma(0)\sigma(t) \rangle$.

3.3 Correction 3: explicit reduced/absolute branch metadata

The code must stop silently labeling all output as "$G(t)$" without saying whether it is reduced or absolute. Even before the full final algorithm is implemented, the corrected version must at minimum emit branch metadata in the output headers.

4 Final Software Architecture

4.1 Semantic layers

The final VELA design has three conceptually distinct layers.

Tensor semantics

This concerns the structure of the input stress file.

- `shear3`: input columns are xy, xz, yz.
- `tensor6`: input columns are xx, yy, zz, xy, xz, yz, internally converted to the three normal-stress differences plus the three shear stresses.

Branch semantics

This concerns what physical modulus can be identified.

- `reduced`: no affine anchor supplied; report reduced modulus only.
- `absolute`: affine modulus supplied externally or computed internally; report absolute modulus.

No separate "legacy" physics branch is kept in the final design. Legacy behavior survives only as an interface compatibility layer.

Transform semantics

This concerns how time-domain data are mapped into frequency-domain observables.

- `direct`
- `i-rheo`
- `schwarzl`
- `caft-bounds`

5 Automatic Branch Determination

The final logic should be automatic and deterministic.

5.1 Stress-only input

If the user provides only a time series of stress components (three or six columns, depending on tensor mode), VELA will automatically choose

$$\texttt{branch = reduced}. \tag{10}$$

This is the safe branch because no plateau anchor is available.

5.2 Stress plus explicit scalar μ_A

If the user supplies stress data and a scalar affine modulus, VELA will automatically choose

$$\texttt{branch = absolute}. \tag{11}$$

In the currently corrected repository stage, this is done using a raw affine modulus in the same scaling convention as the unscaled VELA output.

5.3 Stress plus a time series or block series of μ_A

This also triggers the absolute branch, but with statistical estimation of the affine modulus before propagation to the final outputs.

5.4 Stress plus coordinates/topology/force-field information

If the user supplies stress data together with the files and settings required to reconstruct the force field and instantaneous geometry, VELA should compute μ_A internally and then switch to the absolute branch.

6 μ_A Support Pathways

The final design supports three user-facing pathways.

6.1 Pathway A: scalar μ_A

A single affine modulus value is supplied directly. This is the lightest-weight absolute-branch trigger.

6.2 Pathway B: statistical μ_A data

VELA reads a file containing per-frame or per-block affine modulus values, estimates the mean and uncertainty, and propagates that uncertainty through the absolute-branch outputs.

6.3 Pathway C: internal μ_A computation through libLAMMPS

This is the most important final absolute-branch pathway.

LAMMPS is explicitly designed to be linked as an external library. The official documentation states that LAMMPS can be built as a static or shared library and driven via a C-style library interface.[4]

LAMMPS also provides the `compute born/matrix` command, which returns the second derivatives of the potential energy with respect to strain, i.e. the Born matrix contribution needed in finite-temperature elastic constant calculations. The official documentation and howto page explicitly identify this route as a reliable finite-temperature elastic-constant workflow.[5]

The final VELA μ_A backend should therefore support two libLAMMPS sub-modes:

- `analytic-born`: direct use of `compute born/matrix` when the chosen interaction styles implement analytic strain derivatives.
- `numdiff-born`: automatic fallback to `compute born/matrix numdiff delta virial-ID` when analytic Born derivatives are unavailable.

7 Transform Stack

7.1 Direct transform

The current repository already uses a nontrivial direct transform on the irregular multi-tau time grid. This must be retained as the default fast transform and as a compatibility baseline.

7.2 i-RheoFT baseline

The i-RheoFT method targets the transformation of finite, discretely sampled relaxation functions into storage and loss moduli by analytic Fourier transformation of the sampled representation. It is a natural and necessary external baseline for VELA.[6]

7.3 Schwarzl baseline

Schwarzl-based interconversion remains a classical baseline and is already exposed in RepTate. It should be available in VELA as a standard comparison transform.[7]

[4] LAMMPS documentation, "Use LAMMPS as a library," https://docs.lammps.org/Build_link.html.

[5] LAMMPS documentation, "compute born/matrix," https://docs.lammps.org/compute_born_matrix.html; "Howto elastic," https://docs.lammps.org/Howto_elastic.html.

[6] R. E. O'Connor, M. J. Tassieri, and R. M. L. Evans, "i-RheoFT: Fourier transform-based interconversion of the time and frequency domains for polymeric materials," *Macromolecules* **51**, 8537–8546 (2018), DOI: 10.1021/acs.macromol.8b00447.

[7] RepTate documentation, G(t) application and available views, https://reptate.readthedocs.io/manual/Applications/Gt/general.html.

7.4 Main final method: CAFT-Bounds

The main algorithmic innovation of VELA should be

CAFT-Bounds: Constrained Analytic Fourier Transform with Bounds.

This is the final algorithm name to use in code, papers, and documentation.

8 CAFT-Bounds: Core Algorithm

CAFT-Bounds is designed to outperform direct noisy-tail transformation in exactly the regimes that matter most for MD rheology: finite windows, noisy tails, unknown plateaus, and nonuniform multi-tau grids.

8.1 Step 1: branch-aware input object

CAFT-Bounds never starts from an ambiguously named "$G(t)$" file. It starts from one of:

$$G_{\mathrm{red}}(t), \qquad G_{\mathrm{abs}}(t). \tag{12}$$

The branch is already resolved before the transform step.

8.2 Step 2: trusted-window detection

The current repository uses a $k\sigma$ tail cutoff heuristic. In the final method this heuristic is retained only as a *trusted-window detector*, not as a physical truncation rule.

Given block- or bootstrap-derived error bars σ_i, define a trusted endpoint t_c as the first location beyond which the raw data are no longer statistically distinguishable from noise, e.g.

$$|G(t_i)| \lesssim k\,\sigma_i \tag{13}$$

for a user-controlled threshold k and a short consecutive run criterion.

8.3 Step 3: constrained tail model

For $t > t_c$, CAFT-Bounds does not set the signal to zero. Instead it infers a tail from a physically admissible family.

For the reduced branch:

$$G_{\mathrm{red}}(t) = \sum_{k=1}^{K} g_k e^{-t/\tau_k}, \qquad g_k \geq 0. \tag{14}$$

For the absolute branch:

$$G_{\mathrm{abs}}(t) = G_e + \sum_{k=1}^{K} g_k e^{-t/\tau_k}, \qquad g_k \geq 0, \quad G_e \geq 0. \tag{15}$$

The τ_k are fixed on a logarithmic grid; only the amplitudes and (if needed) the plateau are optimized.

8.4 Step 4: constrained point estimate

The default point estimate is a weighted nonnegative least-squares or quadratic-program problem,

$$\min_x \ \|W(Ax - y)\|_2^2 + \lambda \|Dx\|_2^2, \tag{16}$$

subject to positivity constraints. Here y is the trusted time-domain prefix, A is the time-domain basis operator, W encodes uncertainty, and D is a smoothness operator. This preserves a well-conditioned and reproducible default path before any future Bayesian extension.

8.5 Step 5: bounds

The decisive extra layer is to compute, for each frequency,

$$G'_{\min}(\omega),\ G'_{\max}(\omega), \qquad G''_{\min}(\omega),\ G''_{\max}(\omega) \tag{17}$$

across all admissible tail models consistent with the trusted prefix, uncertainties, branch semantics, and physical constraints.

This is the reason the algorithm is called *CAFT-Bounds*: the output is not just a curve, but a curve together with a statement of frequency-wise identifiability.

8.6 Step 6: analytic transform of prefix + tail

The final frequency-domain result is obtained by analytic transformation of two pieces:

1. the measured trusted prefix on the irregular multi-tau time grid,
2. the constrained exponential tail model.

For a generalized Maxwell tail,

$$G'(\omega) = G_e + \sum_k g_k \frac{\omega^2 \tau_k^2}{1 + \omega^2 \tau_k^2}, \tag{18}$$

$$G''(\omega) = \sum_k g_k \frac{\omega \tau_k}{1 + \omega^2 \tau_k^2}. \tag{19}$$

9 Implementation Roadmap

9.1 Stage 1: immediate corrections (minimal change)

1. Fix mode 1 to use normal-stress differences.
2. Switch the default evaluation path to centered correlations.
3. Introduce explicit branch metadata and a minimal absolute branch via scalar raw μ_A.

9.2 Stage 2: robust branch engine

1. Automatic reduced/absolute branch determination.
2. Stress-only default to reduced branch.
3. External scalar and series μ_A support.

9.3 Stage 3: libLAMMPS μ_A backend

1. Integrate libLAMMPS.
2. Implement analytic Born and numdiff Born paths.
3. Produce a detailed μ_A report with diagnostics.

9.4 Stage 4: transform stack completion

1. Modularize the current direct transform.
2. Add i-RheoFT baseline.
3. Add Schwarzl baseline.
4. Implement CAFT-Bounds v1.

9.5 Stage 5: validation and method paper

1. Synthetic benchmarks: Maxwell fluids, solids with plateau, noisy truncated data.
2. MD benchmarks: at least one fluid/melt case and one permanent-network/solid case.
3. Comparisons against direct, i-RheoFT, Schwarzl, and external workflows such as RepTate.

10 Validation Strategy

The validation plan must be built around the actual failure modes VELA is designed to fix.

10.1 Synthetic tests

At minimum, the following synthetic truth models are required:

1. Single Maxwell fluid.
2. Multi-mode Maxwell fluid.
3. Permanent solid with nonzero plateau:

$$G(t) = G_e + \sum_i g_i e^{-t/\tau_i}. \tag{20}$$

4. Truncated and noisy plateau-bearing signals, where direct tail truncation is known to fail.

10.2 MD tests

The first manuscript does not need vitrimer-specific application yet. It should focus on:

1. a fluid or melt case, where reduced and absolute numerically coincide in practice,
2. a permanent-network or solid-like case, where the reduced/absolute distinction is essential.

11 Why This Design Is Distinctive

VELA does not need to be the broadest rheology suite in existence. It needs to be *uniquely strong* in its target problem class. Its distinctive features are:

1. automatic reduced/absolute branch semantics,
2. integrated μ_A acquisition pathways,
3. four-way transform stack (direct, i-RheoFT, Schwarzl, CAFT-Bounds),
4. low-frequency bounds and trusted-window reporting rather than blind extrapolation.

Taken together, these features make VELA not just a converter, but a branch-aware viscoelastic inference tool.

12 Immediate Deliverables for the Current Development Step

The current code-update step should deliver the following concrete changes on top of the public repository:

1. mode 1 corrected to use normal-stress differences,
2. centered correlations turned on by default,
3. reduced/absolute branch metadata introduced,
4. scalar raw μ_A support added as the first absolute-branch trigger,
5. README updated to describe the corrected semantics,
6. smoke-test and compilation verification performed.

These changes are intentionally minimal but scientifically decisive: they correct the most immediate semantic and implementation errors without yet refactoring the whole codebase.

13 Conclusion

The final vision of VELA is now cleanly defined. It will not simply be a tool that transforms stress autocorrelation functions into viscoelastic curves. Instead, it will be a software framework that:

- identifies whether the input supports a reduced or absolute modulus,
- acquires or computes μ_A when possible,
- provides standard external baselines (direct, Schwarzl, i-RheoFT),
- and deploys CAFT-Bounds as its distinctive main method for trustworthy low-frequency rheology under finite-window uncertainty.

That is the complete target architecture toward which the present corrected repository stage is the first operational step.

References

[1] H. Wu, *VELA public repository*, GitHub, https://github.com/haoyuwu97/VELA.

[2] J. P. Wittmer, H. Xu, O. Benzerara, P. Politi, M. P. Ciamarra, and J. Baschnagel, "Shear-stress relaxation and ensemble transformation of shear-stress autocorrelation functions," *Physical Review E* **91**, 062306 (2015).

[3] S. Kriuchevskyi, J. P. Wittmer, J. Baschnagel, and D. Vlassopoulos, "Computing the shear-stress relaxation modulus from the shear-stress mean-square displacement," *Physical Review E* **93**, 012103 (2016).

[4] R. E. O'Connor, M. Tassieri, and R. M. L. Evans, "i-RheoFT: Fourier transform-based interconversion of the time and frequency domains for polymeric materials," *Macromolecules* **51**, 8537–8546 (2018).

[5] RepTate documentation, G(t) application and views, https://reptate.readthedocs.io/manual/Applications/Gt/general.html.

[6] LAMMPS documentation, "Use LAMMPS as a library," https://docs.lammps.org/Build_link.html.

[7] LAMMPS documentation, "compute born/matrix" and "Howto elastic," https://docs.lammps.org/compute_born_matrix.html, https://docs.lammps.org/Howto_elastic.html.